Q.

A social media application needs to sort the following fixed-length IDs:
324, 125, 456, 218, 102, 389, 275, 147
a)Implement MergeSort and execute it on the given data. Record the contents of the
array after each digit position is processed. b)Implement Quick Sort for the same input. Record the important partition results and
final sorted sequence. c)Analyse MergeSort and Quick Sort using: • Number of passes/partitions
• Comparisons or major operations
• Time complexity
• Additional space
Based on the execution and theoretical analysis, determine which method is more
appropriate for large fixed-length keys.

CODE

```
#include <stdio.h>

void display(int a[], int n)
{
    int i;

    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    printf("\n");
}



int mergeComparisons = 0;
int pass = 1;


void merge(int a[], int low, int mid, int high)
{
    int temp[20];
    int i, j, k;

    i = low;
    j = mid + 1;
```

```
    k = low;

    while (i <= mid && j <= high)
    {
        mergeComparisons++;

        if (a[i] <= a[j])
        {
            temp[k] = a[i];
            i++;
        }
        else
        {
            temp[k] = a[j];
            j++;
        }

        k++;
    }

    while (i <= mid)
    {
        temp[k] = a[i];
        i++;
        k++;
    }

    while (j <= high)
    {
        temp[k] = a[j];
        j++;
        k++;
    }

    for (i = low; i <= high; i++)
    {
        a[i] = temp[i];
    }
}


void mergeSort(int a[], int n)
{
    int size;
    int low, mid, high;
```

```
    size = 1;

    while (size < n)
    {
        low = 0;

        while (low < n - size)
        {
            mid = low + size - 1;
            high = low + 2 * size - 1;

            if (high >= n)
            {
                high = n - 1;
            }

            merge(a, low, mid, high);

            low = low + 2 * size;
        }

        printf("Pass %d: ", pass);
        display(a, n);

        pass++;
        size = size * 2;
    }
}


int quickComparisons = 0;
int partitionNumber = 1;


int partition(int a[], int low, int high)
{
    int pivot;
    int i, j;
    int temp;

    pivot = a[low];

    i = low + 1;
    j = high;
```

```
while (i <= j)
{
    while (i <= high)
    {
        quickComparisons++;

        if (a[i] > pivot)
            break;

        i++;
    }

    while (j >= low)
    {
        quickComparisons++;

        if (a[j] <= pivot)
            break;

        j--;
    }

    if (i < j)
    {
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;

        i++;
        j--;
    }
}

temp = a[low];
a[low] = a[j];
a[j] = temp;

printf("\nPartition %d\n", partitionNumber);
printf("Pivot = %d\n", pivot);
printf("Array after partition: ");
display(a, 8);

partitionNumber++;
```

```
    return j;
}


void quickSort(int a[], int low, int high)
{
    int p;

    if (low < high)
    {
        p = partition(a, low, high);

        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}


int main()
{
    int a[] = {324, 125, 456, 218, 102, 389, 275, 147};
    int b[] = {324, 125, 456, 218, 102, 389, 275, 147};

    int n = 8;

    /* Original Array */

    printf("====================================\n");
    printf("       SORTING ALGORITHM ANALYSIS\n");
    printf("====================================\n");

    printf("\nOriginal Array:\n");
    display(a, n);




    printf("\n========== MERGE SORT ==========\n");

    mergeSort(a, n);

    printf("\nFinal Sorted Array:\n");
    display(a, n);

    printf("Number of comparisons = %d\n", mergeComparisons);
```

```
    printf("\n========== QUICK SORT ==========\n");

    printf("First element is selected as pivot.\n");

    quickSort(b, 0, n - 1);

    printf("\nFinal Sorted Array:\n");
    display(b, n);

    printf("Number of comparisons = %d\n", quickComparisons);


    printf("\n====================================\n");
    printf("            PROGRAM END\n");
    printf("====================================\n");

    return 0;
}
```

OUTPUT

```
==================================
       SORTING ALGORITHM ANALYSIS
==================================

Original Array:
324 125 456 218 102 389 275 147

========== MERGE SORT ==========
Pass 1: 125 324 218 456 102 389 147 275
Pass 2: 125 218 324 456 102 147 275 389
Pass 3: 102 125 147 218 275 324 389 456

Final Sorted Array:
102 125 147 218 275 324 389 456
Number of comparisons = 17

========== QUICK SORT ==========
First element is selected as pivot.

Partition 1
Pivot = 324
Array after partition: 275 125 147 218 102 324 389 456

Partition 2
Pivot = 275
Array after partition: 102 125 147 218 275 324 389 456

Partition 3
Pivot = 102
Array after partition: 102 125 147 218 275 324 389 456

Partition 4
Pivot = 125
Array after partition: 102 125 147 218 275 324 389 456

Partition 5
```

```
Partition 1
Pivot = 324
Array after partition: 275 125 147 218 102 324 389 456

Partition 2
Pivot = 275
Array after partition: 102 125 147 218 275 324 389 456

Partition 3
Pivot = 102
Array after partition: 102 125 147 218 275 324 389 456

Partition 4
Pivot = 125
Array after partition: 102 125 147 218 275 324 389 456

Partition 5
Pivot = 147
Array after partition: 102 125 147 218 275 324 389 456

Partition 6
Pivot = 389
Array after partition: 102 125 147 218 275 324 389 456

Final Sorted Array:
102 125 147 218 275 324 389 456
Number of comparisons = 27
```